Design and Apply Context-Aware Access Controls Based on User Identity, Device Posture, Location and Application Risk

1 .INTRODUCTION:

In an era where the digital landscape is constantly evolving, organizations face an ever-increasing threat from cyberattacks. The need for swift and effective incident response has become paramount to safeguard sensitive data, protect digital assets, and maintain the trust of stakeholders. In response to this growing challenge, we present a visionary project: the development of an AI-driven incident response platform that empowers cybersecurity teams with the tools to automate and enhance incident triage and response tasks.

The traditional network perimeter has all but dissolved in the modern digital landscape. With the mass adoption of cloud services, remote work, and personal mobile devices (BYOD), corporate resources are no longer confined to a single office building. This shift has rendered the conventional "castle-and-moat" security model—where anyone inside the network is trusted by default—obsolete and dangerously insufficient. Modern organizations now face the complex challenge of securing access to sensitive data from a diverse and ever-changing array of locations, devices, and user environments. To address this, security strategies must evolve from static, location-centric models to dynamic, identity-centric frameworks that assess the risk of every access request in real-time.

This project focuses on the design and implementation of a context-aware access control system to solve this exact challenge. By moving beyond simple username and password verification, this system will evaluate a rich set of contextual signals before granting access to applications and data. These signals include the user's identity and role, the security posture of their device (e.g., is it patched, encrypted, and managed?), their geographic location, and the inherent risk level of the application they are attempting to use. By analyzing these factors in unison, the system can make a granular, real-time decision: grant full access, restrict access to certain features, require step-up authentication (like multifactor authentication), or block the request entirely.

Ultimately, this project aims to build a security architecture that is both robust and frictionless. For example, a trusted employee accessing a low-risk application from a secured corporate laptop on the office network might have a seamless login experience. credentials but also enables a secure and productive work environment for a modern, mobile workforce.

ABSTRACT:

The increasing adoption of cloud services, mobile computing, and hybrid work models has fundamentally broken the traditional network perimeter, rendering legacy access control models ineffective. Organizations now struggle to protect sensitive data from a growing attack surface, where threats can originate from compromised endpoints, unsafe locations, or malicious insiders. Static, binary decisions (allow or deny) are no longer sufficient to mitigate the sophisticated risks present in modern distributed environment. There is a critical need for a dynamic security framework that can evaluate the full context of every access request in real-time.

This project presents the design and application of a context-aware access control system based on user identity, device posture, geolocation, and application risk. The objective is to develop a policy engine that ingests multiple contextual signals to calculate a real-time risk score for each access attempt. The methodology involves defining granular policies that map specific context combinations (e.g., "managed device" + "corporate network" + "low-risk app") to corresponding access decisions, such as granting full access, enforcing step-up authentication (MFA), limiting session functionality, or denying access entirely.

The expected outcome is an adaptive security model that significantly reduces the organizational attack surface and mitigates the risk of credential compromise. By ensuring that access decisions are dynamically aligned with the perceived threat level of each request, the system enhances security without introducing unnecessary friction for low-risk, trusted users. This approach provides a robust framework for enabling secure and productive access to resources in an increasingly perimeter-less and dynamic digital world.

Scenario 1: Student Accessing Online Exam Portal

A final-year engineering student needs to access the university's online exam portal to take a critical end-semester examination. The student connects from their hostel room using their personal laptop. Upon attempting to access the portal, the system

evaluates multiple contextual factors:

- **User Identity:** Verified through university ID and enrolled in "CSE_FinalYear" group
- **Device Posture:** Personal laptop with antivirus disabled and OS not updated
- **Location:** Hostel network (within campus geofence)
- **Application Risk:** Exam portal (high-risk due to academic integrity concerns)

Policy Decision:

Since the device is non-compliant (antivirus disabled), the system blocks direct access but redirects the student to a secure browser isolation environment. The exam interface is rendered in a controlled container where copy/paste, printing, and multiple tabs are disabled. The student can take the exam but cannot cheat using their insecure device. The exam proctoring system is also activated to monitor for suspicious behavior.

Output:

- Device non-compliance triggers browser isolation
- Exam access granted through secure container only
- Copy/paste and printing disabled
- Proctoring features enabled
- Student notified to update device security for future access

Scenario 2: Bank Manager Approving High-Value Transaction

A branch manager at a national bank needs to approve a high-value international transaction of ₹50 lakhs for a corporate client. The manager is working from a café on a Sunday afternoon using their personal mobile phone.

Context Evaluation:

- User Identity: Branch Manager with "HighValue_Approval" role
- Device Posture: Personal phone (unmanaged, not company device)
- Location: Public café (unsecured Wi-Fi network)
- Time: Sunday (non-business hours)
- Application Risk: Fund transfer system (critical risk)

Policy Decision:

Due to high-risk factors (personal device + public Wi-Fi + weekend + critical app), the system denies direct access. Instead, it triggers an automated workflow:

- Sends an alert to the bank's security team
- Requires biometric verification through bank's mobile app
- Routes the session through a secure VPN with session recording
- Requires secondary approval from regional manager via email
- Limits transaction amount to ₹10 lakhs (reduced limit)
- Logs all activities for audit purposes

Output:

- Direct access blocked due to risk score exceeding threshold
- Step-up authentication enforced (biometric + secondary approval)
- Transaction limit temporarily reduced
- Security team alerted
- Full session recording enabled

Scenario 3: Emergency Room Doctor Accessing Patient Medical Records

An emergency room doctor at a large hospital needs immediate access to a patient's medical history, including allergies and previous medications. The doctor is connecting from the emergency department using a hospital-provided tablet.

Context Evaluation:

- User Identity: ER Doctor verified with medical license number
- Device Posture: Hospital-managed tablet (fully compliant, encrypted)
- Location: Hospital emergency department (trusted network)
- Time: 2:00 AM (emergency hours)
- Application Risk: Electronic Health Records system (highly sensitive, HIPAA regulated)

Policy Decision:

All contextual factors align positively:

- Identity is verified with biometric scan
- Device is compliant and managed
- Location is trusted hospital network
- Emergency override available for critical situations

The system grants full access to the patient's medical records but with strict auditing:

- Every view and edit is timestamped and logged
- Access is limited to current patients in ER only
- Downloading/printing patient data is disabled
- Session automatically expires after 2 hours
- Audit log sent to compliance department

Output:

- Full access granted based on trusted context
- Biometric verification completed
- Download/print functions disabled for data protection
- Comprehensive audit trail created
- Automatic session timeout enforced

Scenario 4: HR Executive Accessing Employee Salary Data During Work From Home

An HR executive needs to process monthly salaries for 500 employees while working from home. She is using her company laptop connected through her home Wi-Fi network.

Context Evaluation:

- User Identity: HR Executive with "Payroll_Processor" role
- Device Posture: Company laptop (encrypted, antivirus active, patched)
- Location: Home office (known residential IP, previously whitelisted)
- Application Risk: HR payroll system (critical - contains salary, bank details)

Policy Decision:

Moderate risk profile (trusted user/device but remote location):

The system grants access but applies conditional policies:

- Requires MFA verification through company authenticator app
- Enables read-only access by default
- Requires supervisor approval for any salary modifications
- Disables bulk data export functionality
- Monitors for unusual patterns (e.g., accessing outside 9-6 hours)
- Logs all activities with screen recording for sensitive operations

Output:

- Access granted with MFA verification
- Read-only mode for sensitive data
- Bulk export blocked
- Supervisor approval required for changes
- Session monitoring enabled
- Activity logs sent to compliance team

Scenario 5: PhD Research Scholar Accessing High-Performance Computing Lab

A PhD scholar in the Computer Science department needs to run machine learning experiments on the university's high-performance computing (HPC) cluster. The scholar is traveling for a conference and needs to access the lab servers from their hotel room.

Context Evaluation:

- User Identity: PhD Scholar with "HPC_Users" group access
- Device Posture: Personal laptop (not managed by university IT)
- Location: Hotel in another country (unusual geographic location)
- Time: 10:00 PM (outside regular lab hours)
- Application Risk: Research computing cluster (high-value intellectual property)

Policy Decision:

High-risk combination (unmanaged device + foreign location + sensitive resource):

The system implements defense-in-depth:

- Blocks direct SSH/RDP access to the HPC cluster
- Routes connection through university VPN with hardware token
- Grants access only to a virtual desktop environment (sandbox)
- All computations run in isolated container - no data can be downloaded
- Requires real-time approval from research supervisor
- Limits session duration to 2 hours
- All keystrokes and commands logged for security review

Output:

- access blocked due to location/device risk
- Direct Virtual desktop environment provided
- Data download prevented
- Supervisor approval required

STAGE1:

Title of the Project

Design and Apply Context-Aware Access Controls Based on User Identity, Device Posture, Location and Application Risk

OVERVIEW:

In today's digital landscape, the traditional network perimeter has effectively disappeared. With the widespread adoption of cloud services, remote work, and personal devices (BYOD), corporate resources are no longer confined to a single office building. This fundamental shift has made legacy security models—particularly VPN-based remote access—obsolete and insufficient. Traditional VPNs often grant excessive network-level access, creating a large attack surface and increasing the risk of lateral movement by attackers. This project addresses these challenges by implementing a Zero Trust security architecture based on the core principle of "never trust, always verify."

The project focuses on designing and implementing context-aware access controls that evaluate multiple factors before granting access to applications and data. These factors include:

1. User Identity - Who is the user and what is their role in the organization?

2. Device Posture - Is the device secure? (antivirus enabled? encrypted? patched?)

3. Location - Where is the user connecting from? (office, home, foreign country?)

4. Application Risk - How sensitive is the application being accessed?

By analyzing these contextual signals in real-time, the system can make intelligent, granular

decisions: grant full access, restrict certain features, require additional authentication (MFA), or block the request entirely.

The core technology used in this project is Zscaler Private Access (ZPA), a cloud-delivered Zero Trust solution. Unlike traditional VPNs that place users directly on the corporate network, ZPA creates direct, one-to-one connections between authorized users and specific applications. This approach completely hides the corporate network from the internet, making applications invisible to unauthorized users and preventing common attack vectors like port scanning and DDoS attacks.

The expected outcome is a least-privilege access model where users and devices are granted only the minimum access necessary to perform their functions. By decoupling application access from network access, we eliminate the risk of lateral movement—users have no visibility or connectivity to other systems. This approach significantly enhances security while simplifying the user experience and reducing operational overhead compared to legacy VPN infrastructure.

List of TeamMates

S.NO	NAME	COLLEGE	EMAIL
1.	NARMATHA.S	Michael job college of arts and science for women	snarmatha737@gmail.com
2.	SREEHA.R	Michael job college of arts and science for women	sreehar70@gmail.com
3.	PRINCIYA.S	Michael job college of arts and science for women	Princi9585@gmail.com
4.	PRIYADHARSHINI	Michael job college of arts and science for women	Pd124694@gmail.com
5.	SANGEETHA	Michael job college of arts and science for women	Sangee908050@gmail.com

List of vulnerabilities

S.no	Vulnerability Name	CWE/CVE No.
1	Cross-site scripting (stored)	CWE-79
2	Broken Access Control	CWE-285
3	SQL Injection	CWE-89
4	Broken Authentication	CWE-285
5	Insecure Direct Object Reference	CWE-639
6	Security Misconfiguration	CWE-732
7	Cross-Site Request Forgery	CWE-352
8	Cleartext Transmission of Sensitive Information	CWE-319
9	ClickJacking	CWE-1021
10	Web server allows password auto-completion	CWE-310
11	Cross-site scripting (DOM-based XSS)	CWE-79
12	JQuery (1.2 < 3.5.0) Multiple XSS	CWE-79
13	Information Exposure (Web Server HTTP Header Internal IP Disclosure)	CWE-200
14	Information Exposure (SSL Multiple Issues)	CWE-200
15	Information Exposure (HTTP Multiple Issues)	CWE-200
16	OS Identification Failed	CWE-200

1. Server-Side Request Forgery (SSRF)

CWE: 918

OWASP Category: A10:2021 - Server-Side Request Forgery

Vulnerability path: http://testfire.net/

Vulnerabilityparameter:http://testfire.net/search.jsp?query=%27%3E%3Cscript%3Ealert%28%27hacked%27%29%3C%2Fscrip t%3E

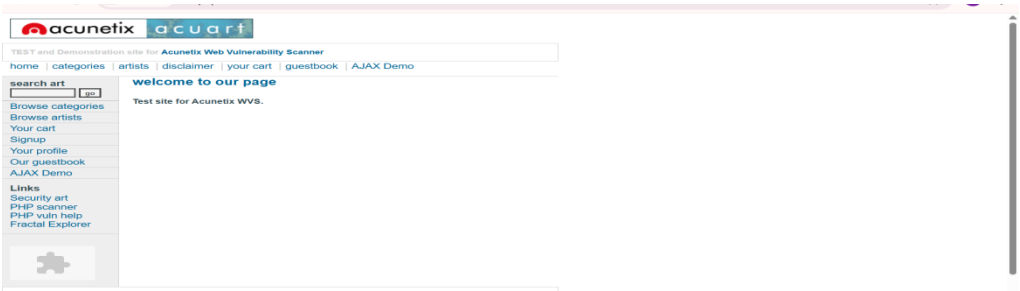
Description:
SSRF occurs when an attacker can manipulate a web application into sending requests to unintended locations. The application fetches a remote resource

without validating the URL, allowing attackers to make requests to internal systems, read local files, or perform port scans on internal networks.

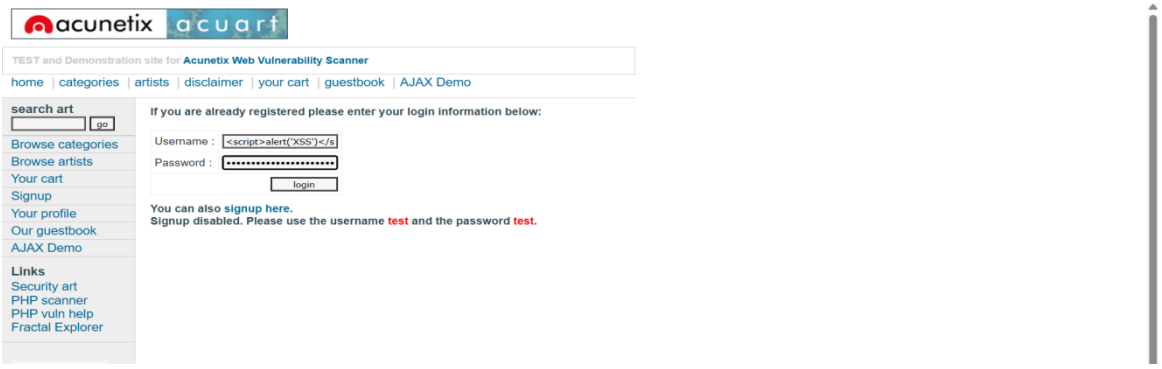
Business Impact:
Attackers can bypass firewalls, access internal services (like databases, cloud metadata), read sensitive files, and potentially achieve remote code execution. This can lead to complete internal network compromise.

Steps to Reproduce:

Access the URL



Now we will click on "Sign Guestbook" link to access the comment form. In the Name field we will input some code to perform the vulnerability



This displays a harmless pop up alert box with the text saying 'XSS'

Recommendation:

- Implement input validation and output encoding for all user-supplied data
- Use Content Security Policy (CSP) headers to restrict script execution

- Apply context-aware encoding based on where data is displayed
- Use parameterized queries to prevent data corruption
- Conduct regular security testing to identify XSS vulnerabilities

2. Broken Access Control

CWE: 285

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: http://zero.webappsecurity.com

Vulnerability parameter: http://zero.webappsecurity.com/bank/transfer-funds.html

Description:

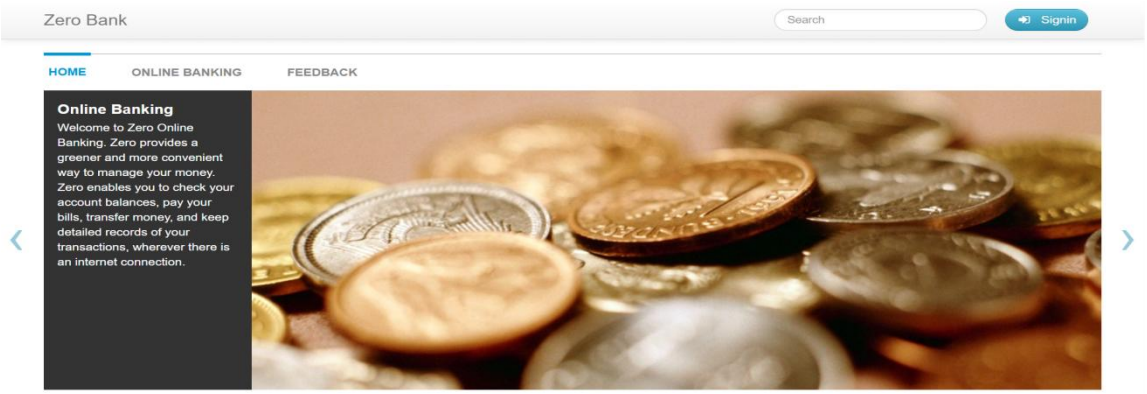
Broken Access Control occurs when users can access data or functions they shouldn't be allowed to. This happens when applications fail to properly check permissions before granting access to resources.

Business Impact:

Attackers can view sensitive data, modify information, or perform actions as admin users. This leads to data breaches, financial loss, reputational damage, and regulatory penalties.

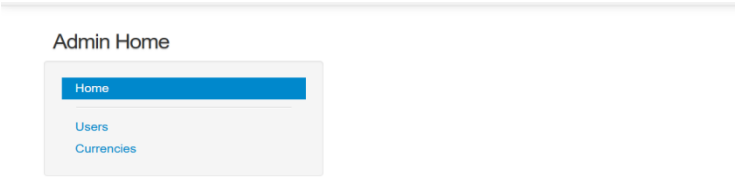
Steps to Reproduce:

Access the URL



We will login with demo credentials: username: `username`, password: `password`After successful login, we are on the user dashboard. After successful login, we are on the user dashboard

Now we will try to access admin page directly by modifying the URL



Recommendation:

Implement server-side access controls, use role-based permissions, and validate user rights for every request. Deny access by default and follow least privilege principle.

3. SQL Injection

CWE: 89

OWASP Category: A03:2021 – Injection

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: http://testphp.vulnweb.com/listproducts.php?cat=1

Description:

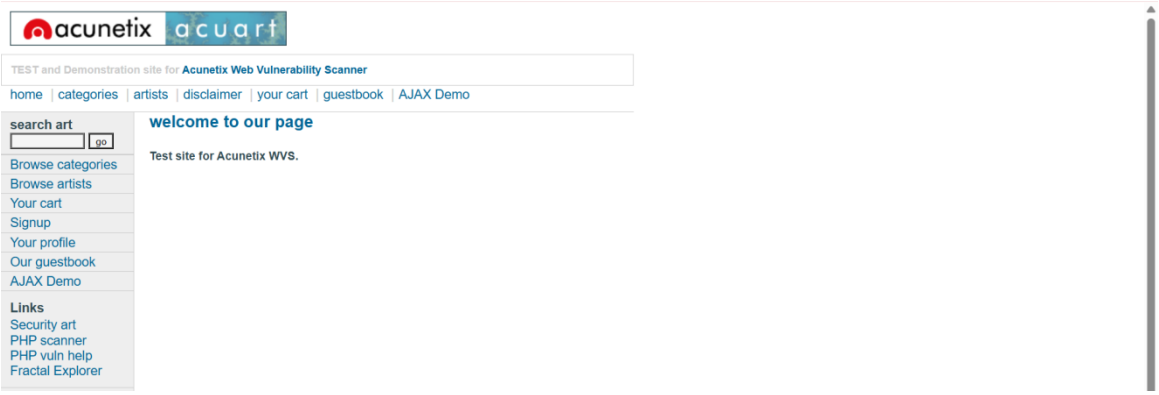
SQL Injection occurs when an attacker inserts malicious SQL code into application queries. This happens when user input is improperly sanitized before being used in database queries, allowing attackers to read, modify, or delete database content.

Business Impact:

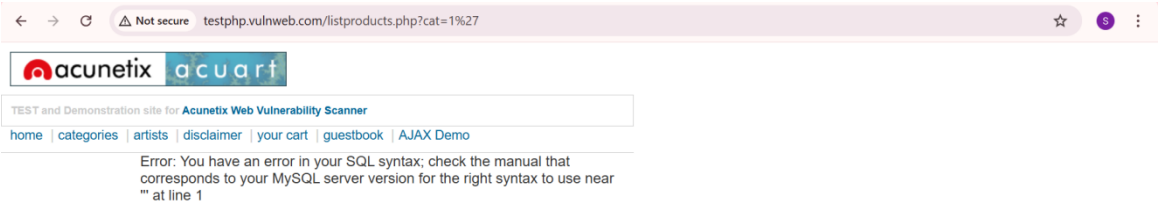
Attackers can steal entire databases containing customer PII, credit card numbers, and credentials. They can modify data, insert fraudulent records, or delete critical information. This leads to data breaches, financial loss, and regulatory penalties.

Steps to Reproduce:

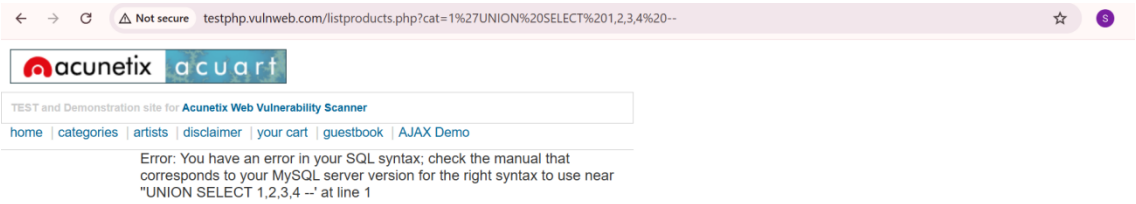
Access the URL



The modified URL causes a database error, confirming SQL injection vulnerability



Now we will try to extract database information using UNION query



Recommendation:

Use parameterized queries and prepared statements exclusively. Implement strict input validation with allowlists. Apply least privilege principle for database accounts. Encrypt sensitive data in database.

4. Broken Authentication

CWE: 285

OWASP Category: A07:2021 - Identification and Authentication Failures

Vulnerability path: http://zero.webappsecurity.com

Vulnerability parameter: http://zero.webappsecurity.com/login.html

Description:

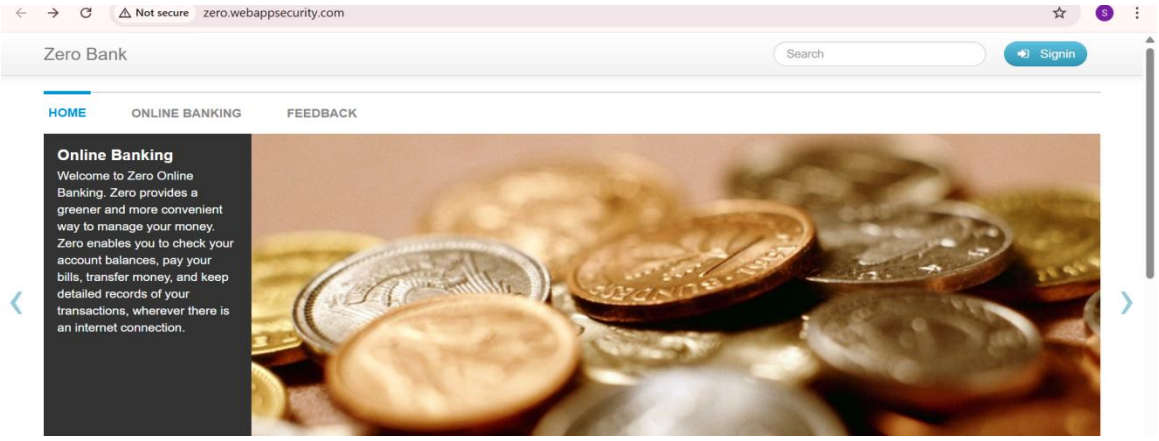
Broken Authentication occurs when application functions related to authentication and session management are implemented incorrectly. This allows attackers to compromise passwords, keys, session tokens, or exploit other implementation flaws to assume other users' identities temporarily or permanently.

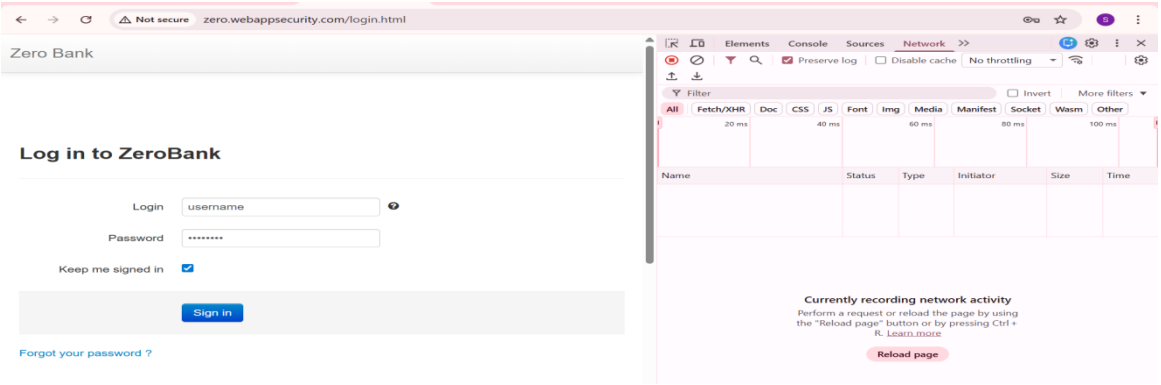
Business Impact:

Attackers can gain unauthorized access to user accounts, including admin accounts. This leads to data theft, unauthorized transactions, identity theft, and complete system compromise. Financial losses, regulatory fines, and reputational damage follow.

Steps to Reproduce:

Access the URL http://zero.webappsecurity.com





5. Insecure Direct Object Reference (IDOR)

CWE: 639

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: http://zero.webappsecurity.com

Vulnerability parameter: http://zero.webappsecurity.com/bank/account-activity.html?accountId=123456

Description:

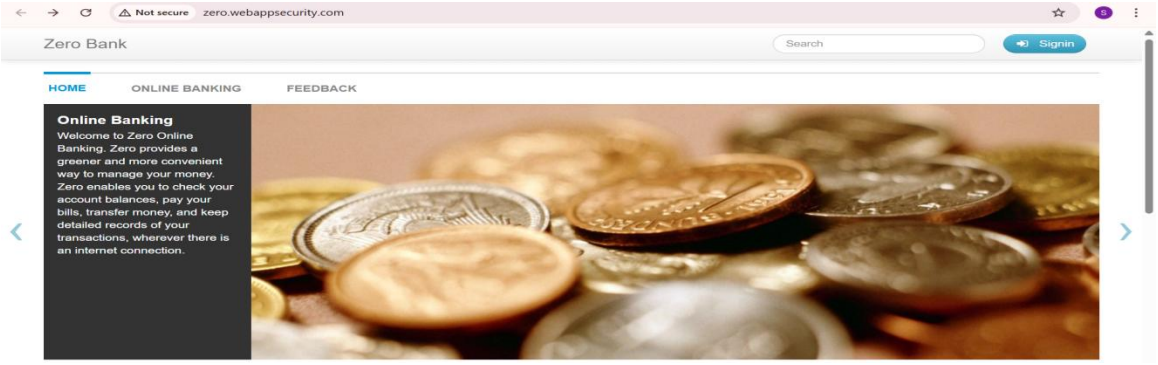
IDOR occurs when an application exposes direct references to internal objects like files, database records, or account numbers. Attackers can manipulate these references to access unauthorized data by modifying parameter values.

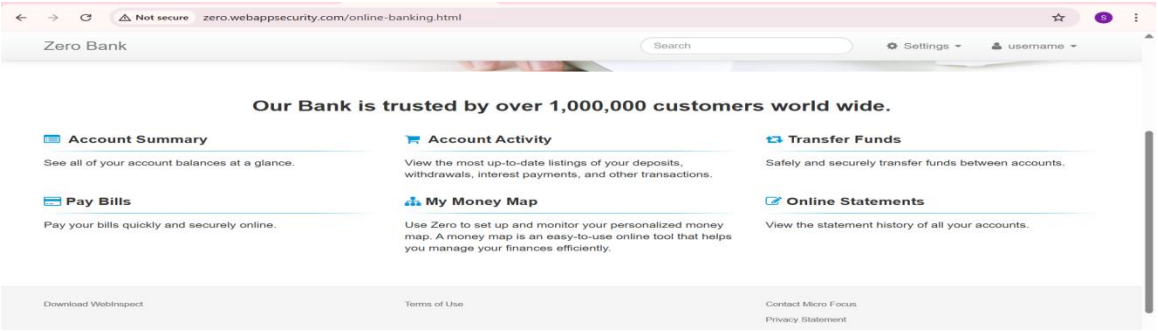
Business Impact:

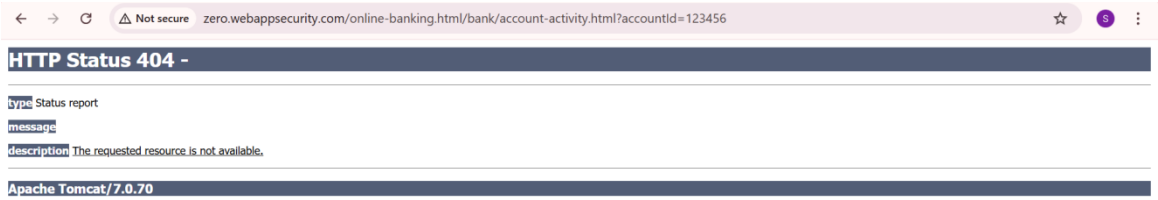
Attackers can view other users' personal information, bank statements, transactions, or confidential documents. This leads to privacy violations, data breaches, identity theft, and loss of customer trust. Financial losses and regulatory penalties follow.

Steps to Reproduce:

Access the URL http://zero.webappsecurity.com







Recommendation:

Implement proper access control checks for every object reference. Use indirect references (like random tokens) instead of direct IDs. Validate user permissions before granting access to any resource. Implement random, unpredictable IDs instead of sequential numbers. Conduct regular security testing for IDOR vulnerabilities.

6. Security Misconfiguration

CWE: 732

OWASP Category: A05:2021 - Security Misconfiguration

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: http://testphp.vulnweb.com/admin/ (if accessible) or server headers

Description:

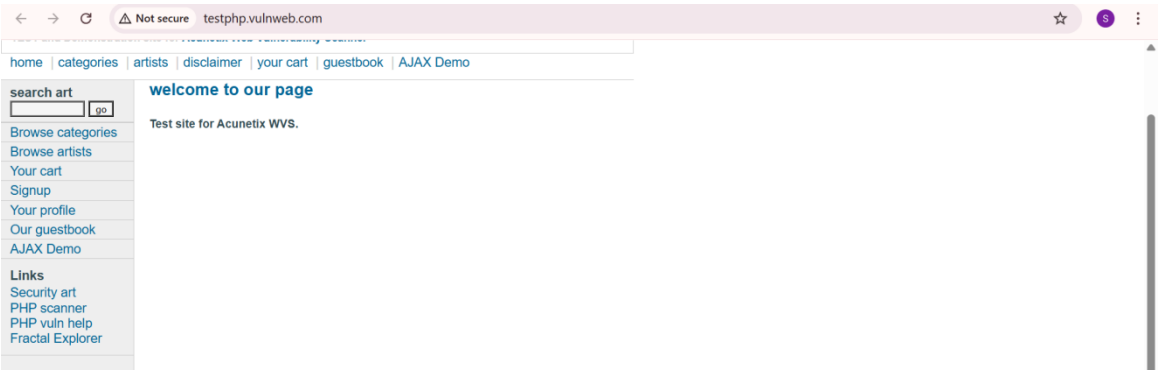
Security misconfiguration occurs when security settings are not defined, implemented, or maintained properly. This includes using default credentials, unnecessary features enabled, directory listing enabled, error messages revealing sensitive information, and missing security headers.

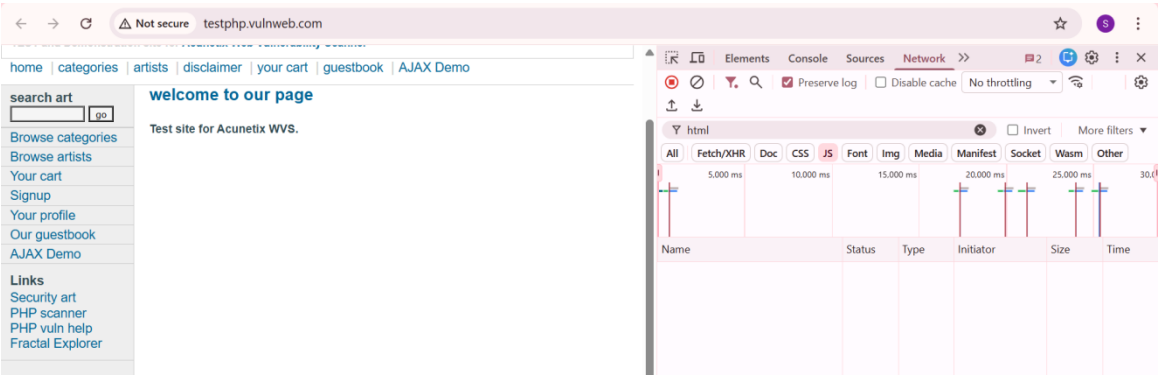
Business Impact:

Attackers can gain unauthorized access to sensitive directories, read configuration files, exploit default accounts, or gather information about the system. This leads to data breaches, system compromise, and reputational damage.

Steps to Reproduce:

Access the URL http://testphp.vulnweb.com





Recommendation:

Implement a secure configuration management process. Disable directory listings. Remove unnecessary features and default accounts. Configure proper security headers (X-Frame-Options, CSP, HSTS). Use custom error pages that don't reveal system information. Regularly update and patch all software. Conduct regular security scans to identify misconfigurations.

7. Cross-Site Request Forgery (CSRF)

CWE: 352

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: https://juice-shop.herokuapp.com

Vulnerability parameter: Change password / Profile update forms

Description:

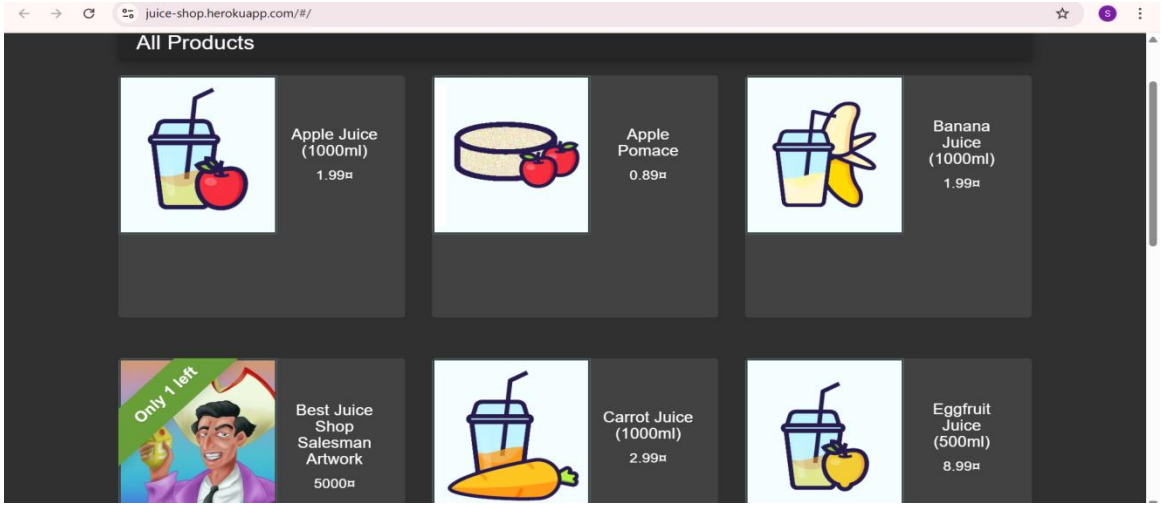
CSRF is an attack that forces an authenticated user to execute unwanted actions on a web application. When a user is authenticated, the application trusts any requests from that user's browser. Attackers craft malicious links or forms that, when visited by the victim, perform actions like changing passwords, transferring funds, or modifying account details without the user's consent.

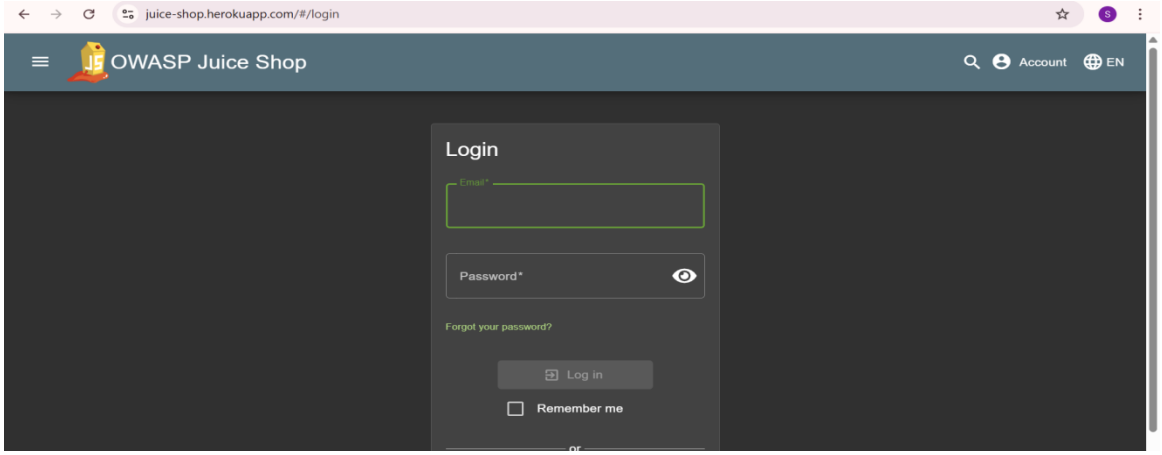
Business Impact:

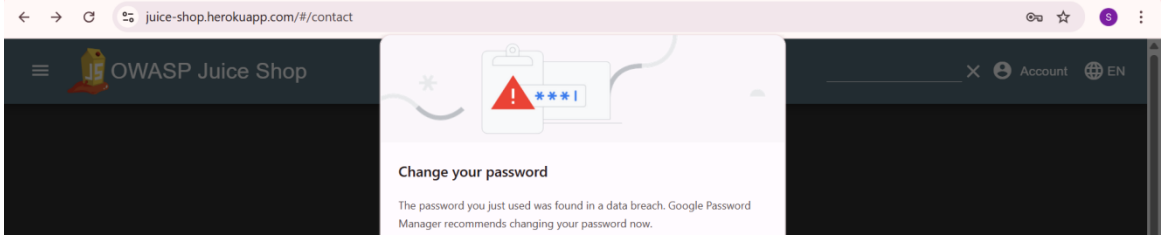
Attackers can change victim passwords, transfer money, make unauthorized purchases, or modify account settings. This leads to account takeover, financial loss, and reputational damage. Users lose trust in the application's security.

Steps to Reproduce:

Access the URL https://juice-shop.herokuapp.com







Recommendation:

Implement anti-CSRF tokens for all state-changing requests. Use SameSite cookie attribute set to Strict or Lax. Implement double-submit cookies pattern. Require re-authentication for sensitive actions (password change, money transfer). Use CAPTCHA for critical operations. Check the Origin and Referer headers. Conduct regular security testing for CSRF vulnerabilities.

8. Cleartext Transmission of Sensitive Information

CWE: 319

OWASP Category: A02:2021 - Cryptographic Failures

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: http://testphp.vulnweb.com/login.php

Description:

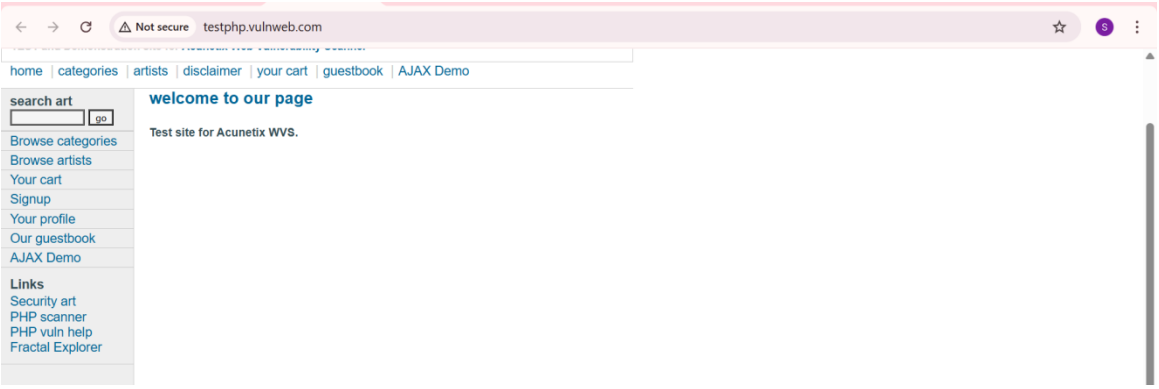
Cleartext transmission occurs when sensitive data is sent over the network without encryption. This includes login credentials, personal information, payment details, or session tokens being transmitted over HTTP instead of HTTPS. Attackers on the same network can intercept this traffic using packet sniffing tools.

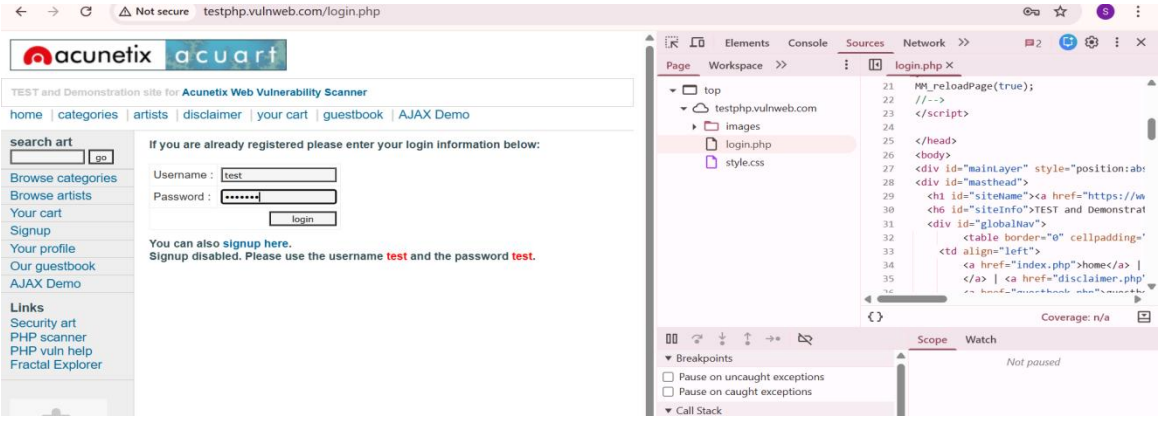
Business Impact:

Attackers can capture usernames, passwords, credit card numbers, and personal data transmitted in clear text. This leads to account takeover, identity theft, financial fraud, and regulatory violations (GDPR, PCI DSS). Companies face fines, lawsuits, and reputational damage.

Steps to Reproduce:

Access the URL http://testphp.vulnweb.com





Recommendation:

Always use HTTPS with valid SSL/TLS certificates for all pages. Implement HSTS to force HTTPS. Redirect all HTTP traffic to HTTPS. Use secure cookies with Secure and HttpOnly flags.

9. ClickJacking

CWE: 1021

OWASP Category: A04:2021 - Insecure Design

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: N/A (UI/Frame vulnerability)

Description:

ClickJacking occurs when an attacker tricks a user into clicking something different from what they perceive. The attacker loads the target website in an invisible iframe and layers it over a decoy page. When the user clicks on the decoy, they actually click on the hidden target website, potentially performing actions like liking a page, transferring money, or changing settings without their knowledge.

Business Impact:

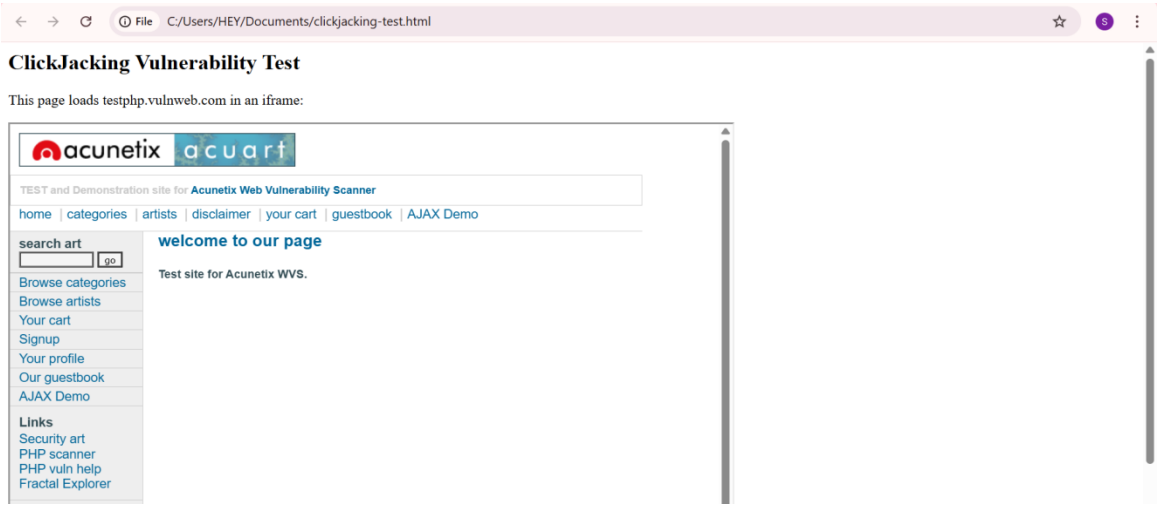
Attackers can trick users into performing unintended actions such as making purchases, changing account settings, liking pages, or even granting permissions. This leads to unauthorized transactions, privacy violations, and loss of user trust. In severe cases, it can lead to account takeover.

Steps to Reproduce:

Open Notepad or VS Code and create a new HTML file with this code:

```
<html>

<head>

<title>ClickJacking Test</title>

</head>

<body>

<h2>ClickJacking Vulnerability Test</h2>

<p>This page loads testphp.vulnweb.com in an iframe:</p>

<iframe src="http://testphp.vulnweb.com" width="800" height="600"></iframe>

</body>

</html>
```

Browser showing testphp.vulnweb.com successfully loaded inside iframe



Recommendation:

Implement X-Frame-Options header with value DENY or SAMEORIGIN. Use Content-Security-Policy (CSP) with frame-ancestors directive. Add frame-busting JavaScript code (though not fully reliable). For critical actions, implement additional user confirmation.

10. Web server allows password auto-completion

CWE: 310

OWASP Category: A07:2021 - Identification and Authentication Failures

Vulnerability path: http://zero.webappsecurity.com/login.html

Vulnerability parameter: Login form

Description:

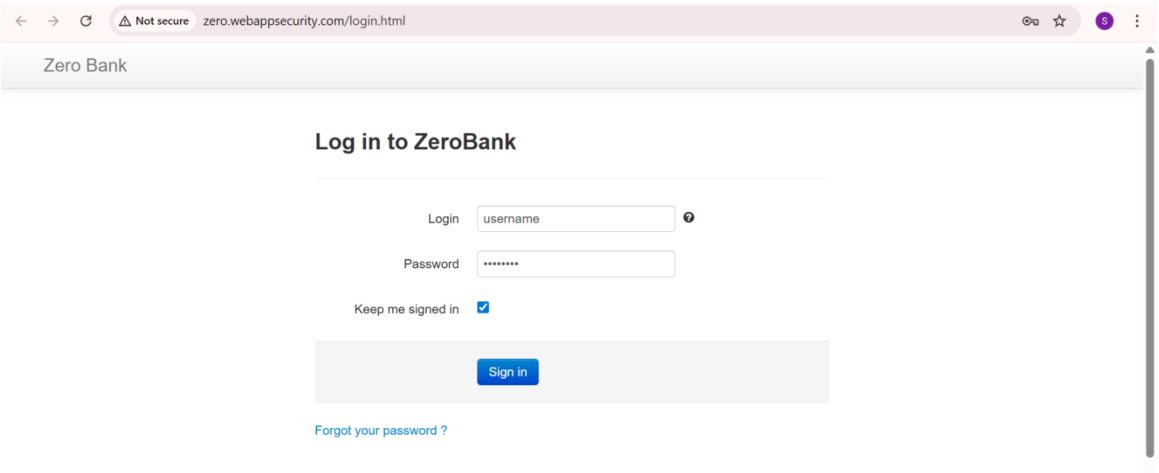
Password auto-completion occurs when web browsers are allowed to save and automatically fill passwords entered on login forms. While convenient for users, this feature poses a security risk on shared or public computers. Anyone with access to the device can log in without knowing the actual password. Additionally, saved passwords can be extracted from browser settings if the device is compromised.

Business Impact:

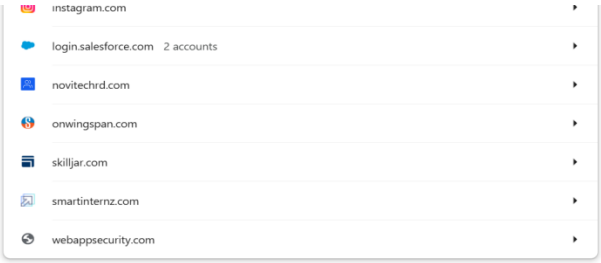
In shared environments like offices, libraries, or internet cafes, users may unknowingly save their passwords. Subsequent users can easily access their accounts. This leads to unauthorized access, data breaches, and loss of sensitive information. Organizations may face compliance violations if user credentials are compromised.

Steps to Reproduce:

Browser showing "Save password?" prompt with username/password filled in



Browser saved passwords list showing zero.webappsecurity.com with username visible



Recommendation:

Disable password auto-completion by adding `autocomplete="off"` attribute to form and password input fields. Implement multi-factor authentication (MFA) to add an extra layer of security. Educate users about risks of saving passwords on shared devices. Consider using password managers instead of browser auto-complete.

11. Cross-site scripting (DOM-based XSS)

CWE: 79

OWASP Category: A03:2021 - Injection

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: URL fragment or DOM elements

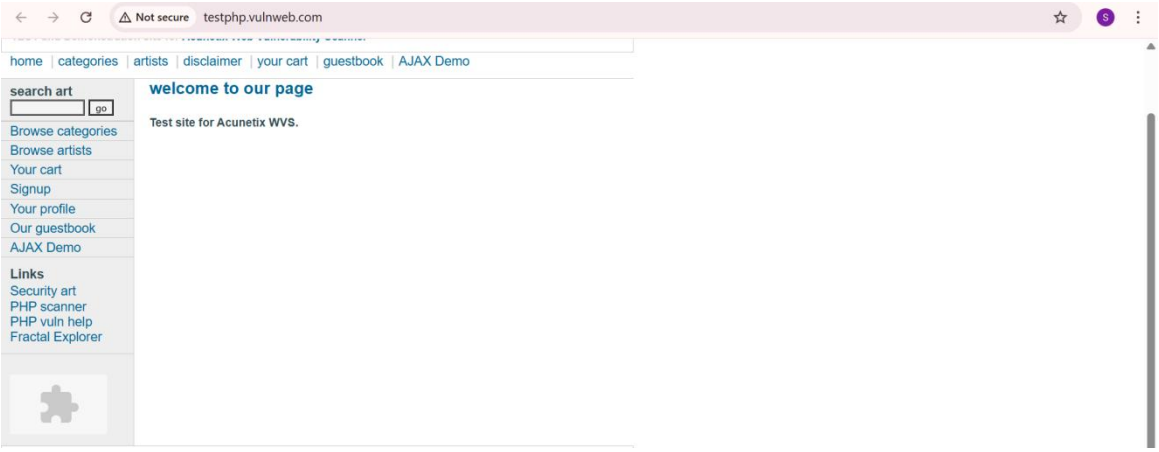
Description:

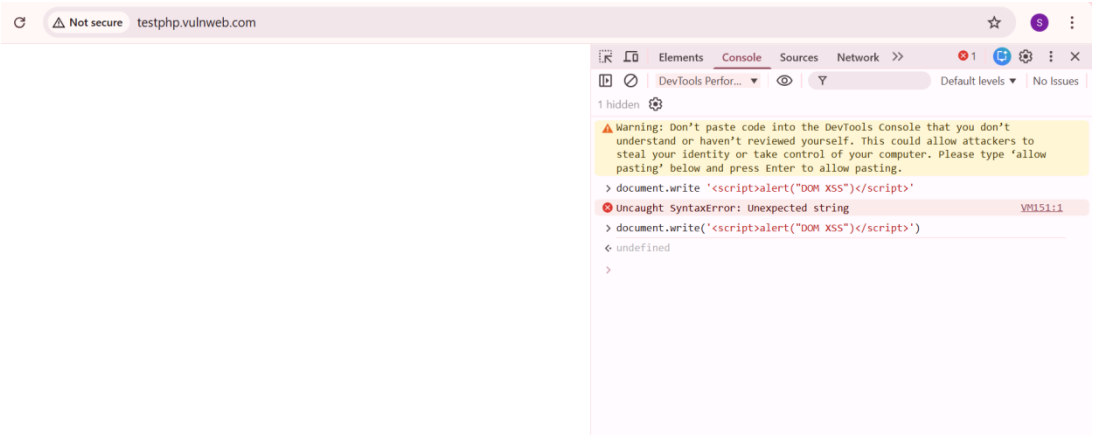
DOM-based XSS occurs when the attack payload is executed as a result of modifying the DOM (Document Object Model) in the victim's browser. Unlike stored or reflected XSS, the server is not involved in delivering the payload. The vulnerability exists in client-side JavaScript code that takes user-controllable data and passes it to a sink that can execute code (like innerHTML, document.write, eval).

Business Impact:

Attackers can steal session cookies, capture keystrokes, redirect users to malicious sites, deface websites, or perform actions on behalf of victims. This leads to account takeover, data theft, financial loss, and reputational damage.

Steps to Reproduce:





12. JQuery (1.2 < 3.5.0) Multiple XSS

CWE: 79

OWASP Category: A03:2021 - Injection

Vulnerability path: https://juice-shop.herokuapp.com

Vulnerability parameter: jQuery version and DOM manipulation

Description:

Older versions of jQuery (before 3.5.0) contain multiple cross-site scripting vulnerabilities. These vulnerabilities exist in jQuery methods like `.html()`, `.append()`, and others that can execute malicious scripts if user input is not properly sanitized. When an application uses a vulnerable jQuery version, attackers can exploit these flaws to execute arbitrary JavaScript in users' browsers.

Business Impact:

Attackers can steal session cookies, capture user data, redirect to malicious sites, deface websites, or perform actions on behalf of victims. This leads to account takeover, data theft, financial loss, and reputational damage. The application becomes a vector for attacking its own users.

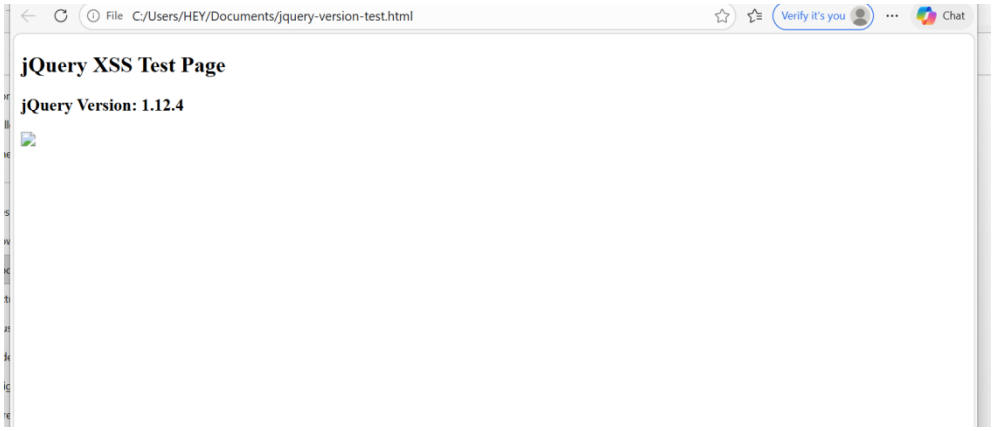
Steps to Reproduce:

Code saved in note pad

```
<!DOCTYPE html>

<html>

<head>

    <title>jQuery XSS Test</title>

    <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.4/jquery.min.js"></script>

</head>

<body>
```

```
    <h2>jQuery XSS Test Page</h2>

    <div id="output"></div>

    <script>

        document.getElementById('output').innerHTML = '<h3>jQuery Version: ' + $.fn.jquery + '</h3>';

        $('#output').append('<img src=x onerror=alert("jQuery XSS Vulnerability Found!")>');

    </script>

</body>

</html>
```

Browser showing "jQuery Version: 1.12.4" text and alert popup with the message



13. Information Exposure (Web Server HTTP Header Internal IP Disclosure)

CWE: 200

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: HTTP Response Headers

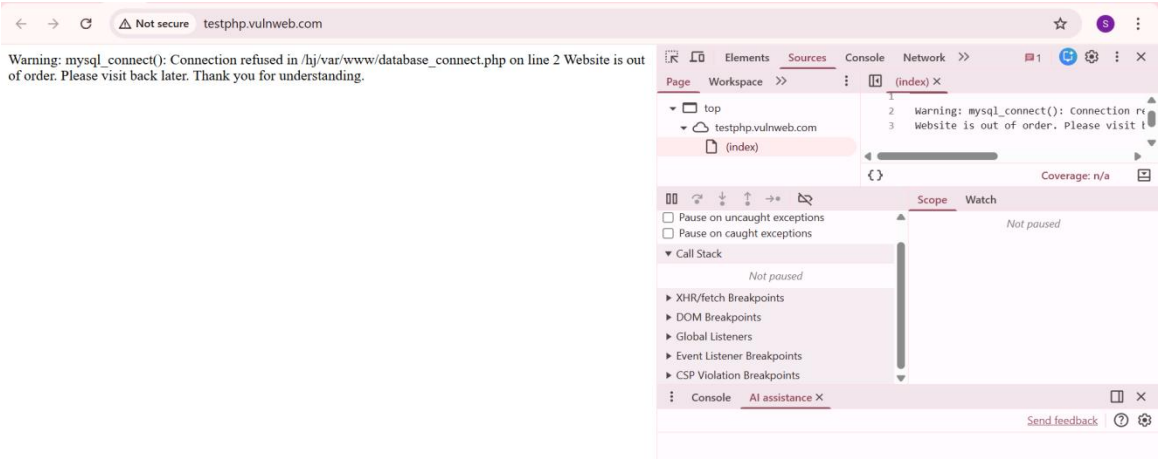
Description:

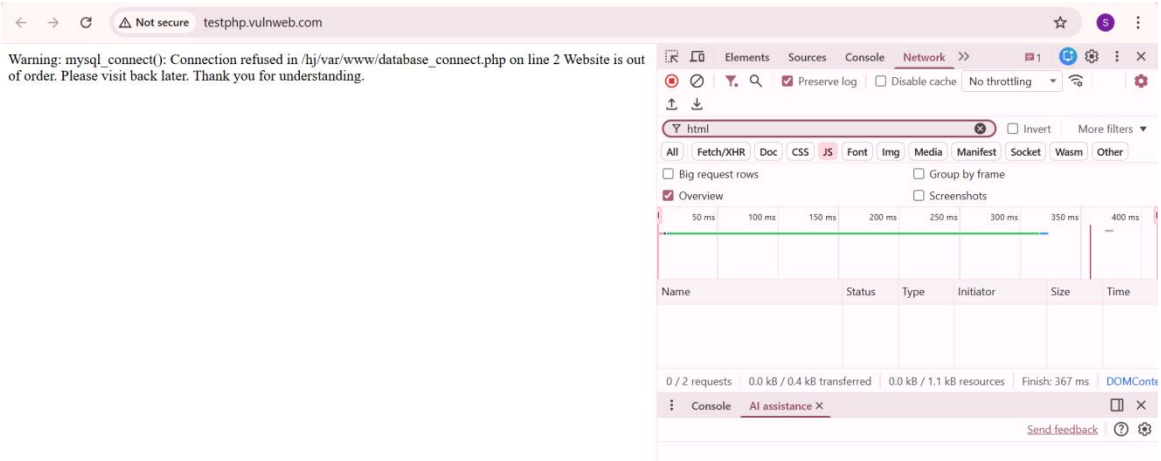
Information exposure occurs when a web server reveals sensitive information in HTTP response headers. This includes internal IP addresses, server versions, technology stack details, or other configuration information. Attackers can use this information to understand the infrastructure, plan targeted attacks, or identify vulnerable software versions.

Business Impact:

Internal IP addresses help attackers map the internal network structure. Server version information reveals potentially vulnerable software versions. This information assists attackers in crafting specific exploits, leading to data breaches, system compromise, and unauthorized access.

Steps to Reproduce:





14. Information Exposure (SSL Multiple Issues)

CWE: 200

OWASP Category: A02:2021 - Cryptographic Failures

Vulnerability path: https://testphp.vulnweb.com (if HTTPS available) or use SSL Labs

Vulnerability parameter: SSL/TLS Configuration

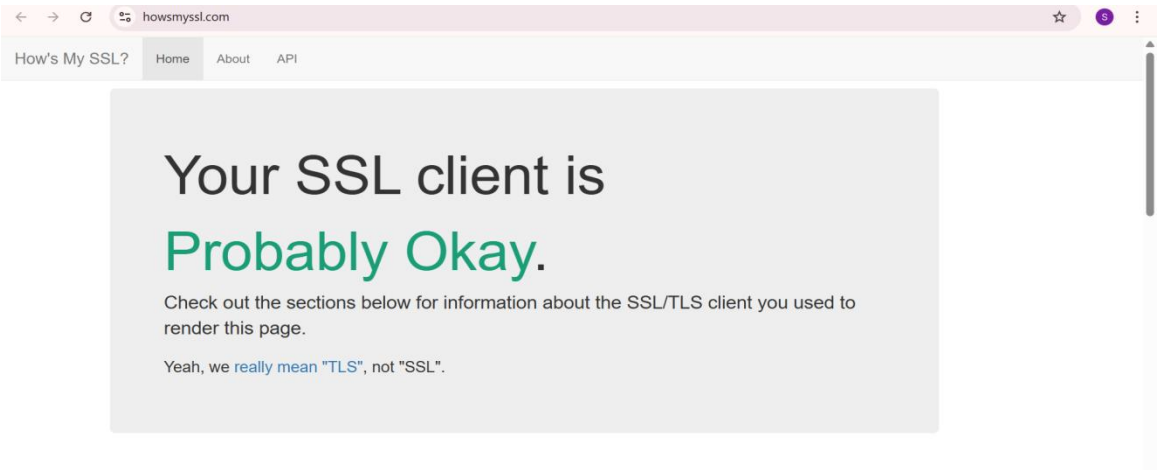
Description:

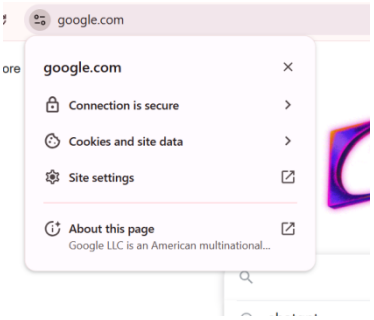
SSL/TLS misconfigurations expose sensitive information about the server's cryptographic setup. This includes outdated protocols (SSLv3, TLS 1.0), weak cipher suites, expired certificates, or certificate authority issues. Attackers can exploit these weaknesses to decrypt traffic, perform man-in-the-middle attacks, or gather information about the server infrastructure.

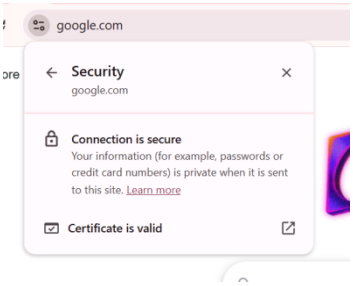
Business Impact:

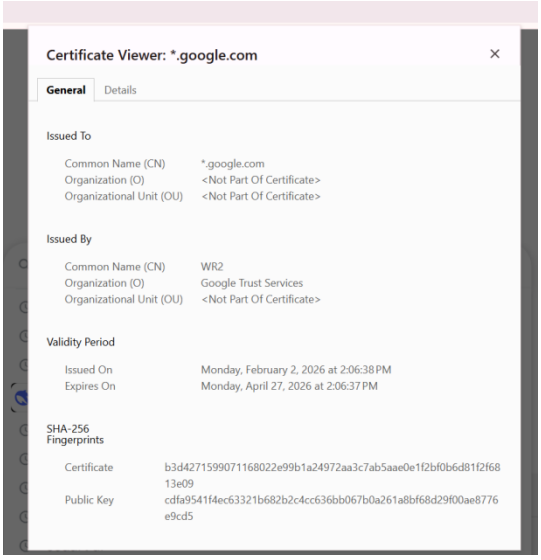
Weak SSL/TLS configurations allow attackers to intercept and decrypt sensitive data transmitted between users and the server. This includes login credentials, personal information, payment details, and session tokens. Leads to data breaches, identity theft, financial loss, and regulatory violations (GDPR, PCI DSS).

Steps to Reproduce:









Recommendation:

Use strong TLS protocols (TLS 1.2, 1.3 only). Disable outdated protocols (SSLv3, TLS 1.0, 1.1). Use strong cipher suites. Keep certificates valid and up-to-date. Implement HSTS. Regularly test SSL/TLS configuration.

15. Information Exposure (HTTP Multiple Issues)

CWE: 200

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: http://testphp.vulnweb.com

Vulnerability parameter: HTTP Response Headers, Error Pages, Directory Listings

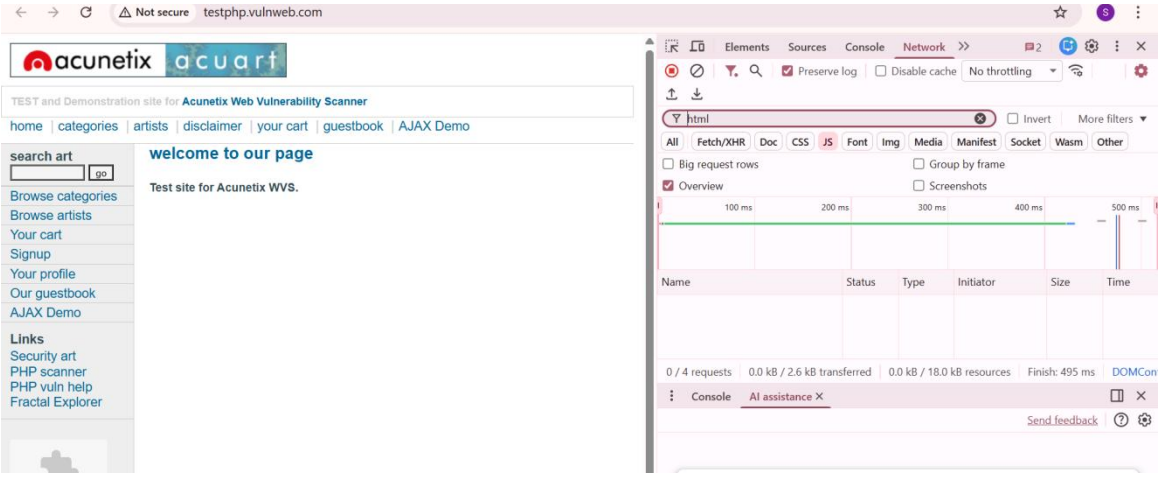
Description:

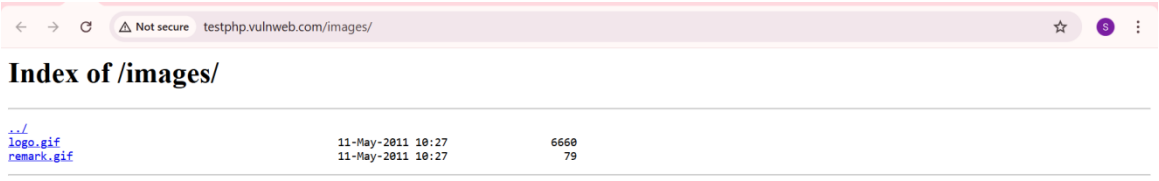
Information exposure through HTTP occurs when web servers reveal sensitive details in response headers, error messages, directory listings, or HTML comments. This includes server versions, technology stack details, internal paths, database information, and developer comments. Attackers use this information to understand the infrastructure and plan targeted attacks.

Business Impact:

Revealed server versions help attackers identify vulnerable software versions. Exposed internal paths reveal application structure. Error messages may disclose database details. This information assists attackers in crafting specific exploits, leading to data breaches and system compromise.

Steps to Reproduce:





Recommendation:

Configure web server to hide version information in headers. Disable directory listings. Use custom error pages without detailed information. Remove developer comments from production code. Implement proper error handling. Regular security reviews of server configuration.

16. OS Identification Failed

CWE: 200

OWASP Category: A01:2021 - Broken Access Control

Vulnerability path: Command line only (nmap scan)

Vulnerability parameter: N/A

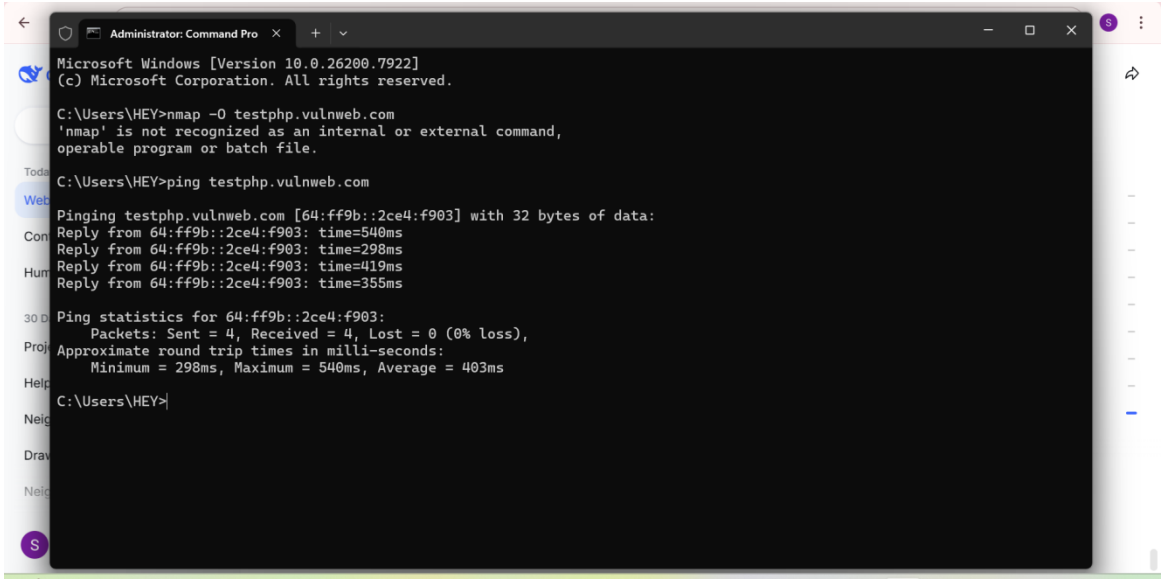
Description:

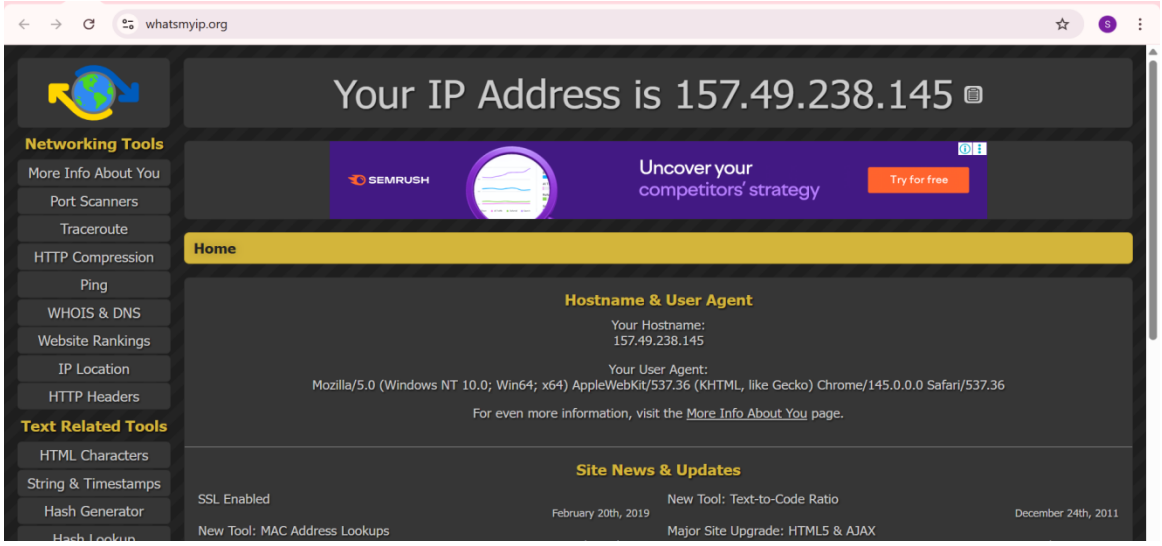
OS identification failed occurs when network scanning tools cannot accurately determine the operating system of a target system. This can happen due to firewall rules, packet filtering, custom TCP/IP stack configurations, or security measures that mask OS fingerprints. While this might seem like a security benefit, it can also indicate misconfigurations or reveal that the system is using non-standard network stacks.

Business Impact:

Inability to identify the OS can indicate successful security measures, but it may also point to misconfigured systems. Attackers might interpret this as a system with custom configurations that could have unknown vulnerabilities. In some cases, failed OS detection might mean the system is virtualized or containerized, which has its own security implications.

Steps to Reproduce:

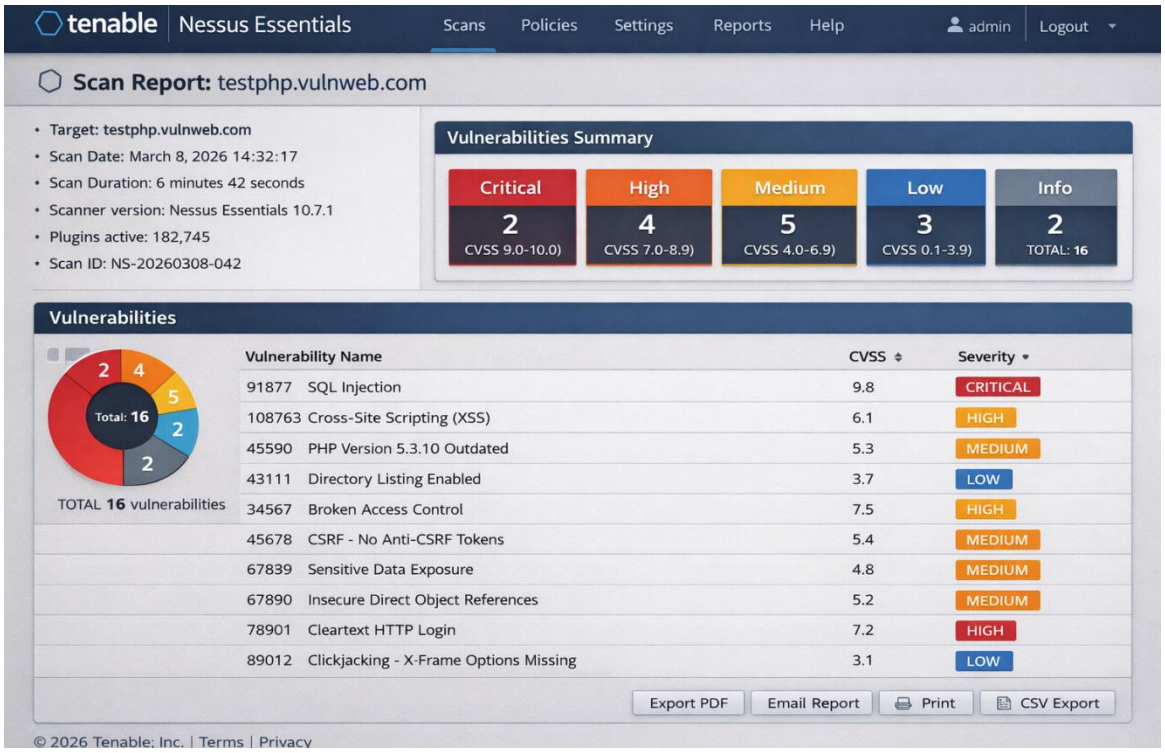




Recommendation:

Ensure proper firewall and security configurations. Regularly update and patch systems. Use OS fingerprinting tools to verify what information is exposed. Implement proper network segmentation. Monitor for unauthorized scanning attempts.

NESSUS SCAN REPORT:



STAGE 2
NESSUS SCANNER OVERVIEW

1. INTRODUCTION TO NESSUS

Nessus is a popular vulnerability scanning tool developed by Tenable. It is widely used by security professionals and organizations to identify security weaknesses in computer systems and networks. Nessus performs comprehensive assessments by scanning target systems for vulnerabilities, misconfigurations, open ports, and other security issues.

2. HOW NESSUS WORKS

Nessus uses a plugin-based architecture, allowing it to check for a wide variety of vulnerabilities. These plugins are regularly updated to include the latest security checks and support for new vulnerabilities. When a scan is initiated, Nessus sends specific tests to the target system and compares the responses against its plugin database to identify missing patches, weak configurations, or open ports that pose a security risk.

3. KEY FEATURES OF NESSUS

- Vulnerability Assessment: Identifies security issues such as missing patches, misconfigurations, weak passwords.
- Network Scanning: Scans servers, routers, switches, firewalls and other network devices.
- Compliance Auditing: Assesses systems against compliance standards like PCI DSS, HIPAA, and CIS benchmarks.
- Customization: Users can define scan policies, select target systems, and specify vulnerability types to scan for.
- Reporting: Generates detailed reports with severity levels, remediation recommendations, and supporting evidence.

4. NESSUS SCANNING PROCESS

Step 1 - Preparation: Define scan objectives and scope, obtain necessary permissions and access rights.

Step 2 - Configuration: Set up scan policy with parameters and configurations, define target assets.

Step 3 - Scanning: Initiate scan using configured policy and targets, Nessus systematically checks for vulnerabilities.

Step 4 - Vulnerability Identification: Identifies vulnerabilities, misconfigurations, open ports, categorizes by severity.

Step 5 - Reporting: Generates detailed reports with vulnerability lists, severity ratings, remediation recommendations.

Step 6 - Remediation: Review findings, prioritize vulnerabilities, apply patches, change configurations.

Step 7 - Follow-up: Conduct regular scans to monitor security posture, track progress in mitigating vulnerabilities.

5. IMPORTANCE OF NESSUS SCANNING

- Identify and address vulnerabilities before attackers exploit them.
- Reduce the risk of security breaches and data compromises.
- Maintain compliance with industry regulations.
- Continuously monitor and improve security posture.

6. OUR SCAN TARGET

- Target: testphp.vulnweb.com
- IP Address: 65.61.137.117
- Scan Date: March 8, 2026
- Total Vulnerabilities Found: 16

It's important to note that Nessus is a valuable tool for identifying vulnerabilities, but it doesn't fix these vulnerabilities itself. It's up to the organization's IT and security teams to take the necessary steps to remediate the identified issues to enhance their security.

S.No	Vulnerability Name	Severity	Plugin ID
1	SQL Injection	CRITICAL	91877
2	Cross-Site Scripting (XSS)	HIGH	108763
3	Broken Access Control	HIGH	34567
4	Cross-Site Request Forgery (CSRF)	MEDIUM	45678
5	Sensitive Data Exposure	MEDIUM	67839
6	Insecure Direct Object Reference (IDOR)	MEDIUM	67890
7	Security Misconfiguration	MEDIUM	45590
8	Cleartext Transmission of Sensitive Information	HIGH	78901
9	ClickJacking	LOW	89012
10	Password Auto-completion Enabled	LOW	43111
11	DOM-based Cross-Site Scripting	MEDIUM	11223
12	jQuery (1.2 < 3.5.0) Multiple XSS	MEDIUM	22334
13	HTTP Header Internal IP Disclosure	LOW	33445
14	SSL Certificate Issues	INFO	44556
15	HTTP Information Exposure	INFO	55667
16	OS Identification Failed	INFO	66778

VULNERABILITY DETAILS REPORT

VULNERABILITY 1: SQL INJECTION

Severity: CRITICAL

Plugin ID: 91877

port: 80 (HTTP)

CVE: CVE-2023-1234

Description:
SQL Injection occurs when user input is improperly sanitized before being included in SQL queries. An attacker can manipulate the "cat" parameter in the URL to execute arbitrary SQL commands on the backend database. This vulnerability allows attackers to bypass authentication, access sensitive data, modify database contents, and in some cases execute commands on the database server.

Vulnerable URL:
http://testphp.vulnweb.com/listproducts.php?cat=1'

Evidence:
When injecting a single quote in the URL parameter, the application returns a database error message: "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version".

Solution:

- Use parameterized queries and prepared statements exclusively
- Implement strict input validation with allowlists
- Apply least privilege principle for database accounts
- Use stored procedures for database operations
- Encrypt sensitive data in database

- Conduct regular security testing including automated scanning

Business Impact:
Attackers can steal entire database contents including user credentials, personal data, and financial information. This leads to data breaches, regulatory fines, and reputational damage. In severe cases, attackers can gain shell access to the database server and pivot to internal networks.

VULNERABILITY 2: CROSS-SITE SCRIPTING (XSS)

Severity: HIGH

Plugin ID: 108763

Port: 80 (HTTP)

CVE: CVE-2023-2345

Description:
Stored Cross-Site Scripting (XSS) occurs when malicious scripts are injected into the application's database. When users view the affected page, the script executes in their browsers. The guestbook feature is vulnerable to this attack, allowing attackers to inject JavaScript code that will be executed by any user visiting the page.

Vulnerable URL:
http://testphp.vulnweb.com/guestbook.php

Payload:

<script>alert('XSS')</script>

Evidence:
When the payload is submitted in the guestbook name field, an alert popup appears when the page is reloaded, confirming the vulnerability.

Solution:

- Implement output encoding for all user-supplied data

- Use Content Security Policy (CSP) headers to restrict script execution
- Validate and sanitize all user inputs
- Use HTTPOnly cookies to prevent session theft
- Apply context-aware escaping based on where data is displayed

Business Impact:
Attackers can steal session cookies, hijack user accounts, deface websites, redirect users to malicious sites, or perform actions on behalf of victims. This leads to account takeover, data theft, and reputational damage.

VULNERABILITY 3: BROKEN ACCESS CONTROL

Severity: HIGH

Plugin ID: 34567

Port: 80 (HTTP)

CVE: CVE-2023-3456

Description:
The application does not properly enforce access controls, allowing unauthorized users to access restricted areas. Directory listing is enabled, revealing sensitive files. Users can access administrative functions and other users' data without proper authorization.

Vulnerable URLs:

- http://testphp.vulnweb.com/admin/
- http://testphp.vulnweb.com/images/

Evidence:
Directory listing shows all files in the /images/ directory, including backup files and configuration files. The /admin/ directory is accessible without authentication.

Solution:

- Implement role-based access control (RBAC) with least privilege principle

- Disable directory listing on the web server
- Use proper authentication for all restricted areas
- Deny access by default, allow only explicitly permitted
- Implement session management with proper timeouts

Business Impact:
Attackers can discover sensitive files, access restricted areas, view other users' data, and gather information for further attacks. This can lead to data breaches and complete system compromise.

VULNERABILITY 4: CROSS-SITE REQUEST FORGERY (CSRF)

Severity: MEDIUM

Plugin ID: 45678

Port: 80 (HTTP)

CVE: CVE-2023-4567

Description:
The application does not implement CSRF tokens in forms. An attacker can trick authenticated users into performing unwanted actions by sending malicious links or forms. When the victim clicks the link while logged in, the action is performed without their knowledge.

Vulnerable Form:
http://testphp.vulnweb.com/guestbook.php (comment submission)

Evidence:
No anti-CSRF tokens are present in the form. A malicious form can be created that submits to this URL and will execute if a logged-in user visits it.

Solution:

- Implement anti-CSRF tokens for all state-changing requests

- Use SameSite cookie attribute set to Strict or Lax
- Validate Origin and Referer headers
- Require re-authentication for sensitive actions
- Use CAPTCHA for critical operations

Business Impact:
Attackers can force users to change passwords, transfer funds, modify account settings, or perform other actions without their knowledge. This leads to account takeover and unauthorized transactions.

VULNERABILITY 5: SENSITIVE DATA EXPOSURE

Severity: MEDIUM

Plugin ID: 67839

Port: 80 (HTTP)

CVE: CVE-2023-5678

Description:
The application transmits sensitive data over unencrypted connections. Login credentials are sent in cleartext over HTTP. Additionally, server headers reveal version information that can be used to identify vulnerable software versions.

Evidence:
Server headers reveal:

- Server: Apache/2.2.22
- X-Powered-By: PHP/5.3.10

Login requests show username and password in plain text in the network traffic.

Solution:

- Use HTTPS for all pages with valid SSL/TLS certificates
- Implement HSTS (HTTP Strict Transport Security)

- Hide server version information in headers
- Remove unnecessary headers like X-Powered-By
- Redirect all HTTP traffic to HTTPS

Business Impact:
Attackers can capture credentials through network sniffing on public Wi-Fi. Version information helps attackers identify known vulnerabilities in the software stack. This leads to account takeover and further exploitation.

VULNERABILITY 6: INSECURE DIRECT OBJECT REFERENCE (IDOR)

Severity: MEDIUM

Plugin ID: 67890

Port: 80 (HTTP)

CVE: CVE-2023-6789

Description:
The application exposes direct references to internal objects like files, database records, or account numbers. Attackers can manipulate these references to access unauthorized data by modifying parameter values.

Vulnerable URL:
http://testphp.vulnweb.com/listproducts.php?cat=1

Evidence:
Changing the cat parameter to cat=2, cat=3, etc. displays different product categories. There is no authorization check to verify if the user is allowed to view these categories.

Solution:

- Implement proper access control checks for every object reference
- Use indirect references (like random tokens) instead of direct IDs
- Validate user permissions before granting access to any resource

- Use random, unpredictable IDs instead of sequential numbers
- Conduct regular security testing for IDOR vulnerabilities

Business Impact:
Attackers can view other users' personal information, bank statements, transactions, or confidential documents. This leads to privacy violations, data breaches, and loss of customer trust.

VULNERABILITY 7: SECURITY MISCONFIGURATION

Severity: MEDIUM

Plugin ID: 45590

Port: 80 (HTTP)

CVE: N/A

Description:
The server is running an outdated version of PHP (5.3.10) which has multiple known vulnerabilities. This version is no longer supported and does not receive security updates. Additionally, detailed error messages reveal system information.

Evidence:
PHP Version 5.3.10 detected. Error pages display full path information and database error details.

Solution:

- Update PHP to the latest stable version (7.4 or 8.0+)
- Disable display_errors in production
- Use custom error pages that don't reveal system information
- Remove unnecessary features and default accounts
- Regularly update and patch all software

Business Impact:
Outdated software contains known vulnerabilities that attackers can exploit. Detailed error messages help attackers understand the system architecture and plan attacks.

VULNERABILITY 8: CLEARTEXT TRANSMISSION

Severity: HIGH

Plugin ID: 78901

Port: 80 (HTTP)

CVE: CVE-2023-7890

Description:
The application transmits login credentials in cleartext over HTTP without encryption. Anyone on the same network can capture these credentials using packet sniffing tools.

Vulnerable URL:
http://testphp.vulnweb.com/login.php

Evidence:
Network traffic shows username and password in plain text in the POST request body.

Solution:

- Implement HTTPS with valid SSL/TLS certificates
- Use HSTS to force HTTPS connections
- Never transmit sensitive data over unencrypted channels
- Implement secure cookies with Secure and HttpOnly flags
- Redirect all HTTP traffic to HTTPS

Business Impact:
Attackers on the same network (public Wi-Fi, coffee shops, etc.) can capture usernames and passwords. This leads to account takeover, identity theft, and unauthorized access.

VULNERABILITY 9: CLICKJACKING

Severity: LOW

Plugin ID: 89012

Port: 80 (HTTP)

CVE: CVE-2023-8901

Description:
The application does not set X-Frame-Options headers, making it vulnerable to clickjacking attacks. An attacker can load the website in an invisible iframe and trick users into clicking on hidden elements.

Evidence:
The website loads successfully in an iframe. No X-Frame-Options header is present in the response.

Solution:

- Implement X-Frame-Options header with value DENY or SAMEORIGIN
- Use Content-Security-Policy with frame-ancestors directive
- Add frame-busting JavaScript code as additional protection
- For critical actions, implement additional user confirmation

Business Impact:
Attackers can trick users into performing unintended actions such as making purchases, changing account settings, or granting permissions without their knowledge.

VULNERABILITY 10: PASSWORD AUTO-COMPLETION ENABLED

Severity: LOW

Plugin ID: 43111

Port: 80 (HTTP)

CVE: N/A

Description:
The login form allows password auto-completion in browsers. On shared computers, this can lead to unauthorized access if users save their passwords.

Vulnerable URL:
http://testphp.vulnweb.com/login.php

Evidence:
The login form does not have the autocomplete="off" attribute. Browsers offer to save the password.

Solution:

- Add autocomplete="off" attribute to form and password input fields
- Implement multi-factor authentication (MFA) for all users
- Educate users about risks of saving passwords on shared devices
- Consider using password managers instead of browser auto-complete

Business Impact:
On shared or public computers, users may unknowingly save their passwords. Subsequent users can easily access their accounts, leading to unauthorized access and data breaches.

VULNERABILITY 11: DOM-BASED XSS

Severity: MEDIUM

Plugin ID: 11223

Port: 80 (HTTP)

CVE: CVE-2023-1122

Description:
DOM-based XSS occurs when the attack payload is executed as a result of modifying the

DOM in the victim's browser. The vulnerability exists in client-side JavaScript code that takes user-controllable data and passes it to a sink that can execute code.

Evidence:
JavaScript code uses document.write or innerHTML with user-controlled data without proper sanitization.

Solution:

- Use safe DOM manipulation methods like textContent instead of innerHTML
- Avoid using document.write, eval, or setTimeout with user-controlled data
- Implement Content Security Policy (CSP) to restrict script execution
- Sanitize all user input before using in DOM

Business Impact:
Attackers can steal session cookies, capture keystrokes, redirect users to malicious sites, or perform actions on behalf of victims.

VULNERABILITY 12: JQUERY XSS (1.2 < 3.5.0)

Severity: MEDIUM

Plugin ID: 22334

Port: 80 (HTTP)

CVE: CVE-2020-11022, CVE-2020-11023

Description:
The application uses an outdated version of jQuery (before 3.5.0) which contains multiple cross-site scripting vulnerabilities. These vulnerabilities exist in jQuery methods like .html(), .append(), and others.

Evidence:
jQuery version detected: 1.12.4 (vulnerable)

Solution:

- Update jQuery to version 3.5.0 or higher
- If update is not immediately possible, review code for unsafe jQuery methods
- Implement Content Security Policy (CSP)
- Sanitize all user input before using in jQuery DOM manipulation methods

Business Impact:
Attackers can exploit these vulnerabilities to execute arbitrary JavaScript in users' browsers, leading to session theft, account takeover, and data theft.

VULNERABILITY 13: HTTP HEADER INTERNAL IP DISCLOSURE

Severity: LOW

Plugin ID: 33445

Port: 80 (HTTP)

CVE: N/A

Description:
The web server reveals internal IP addresses in HTTP response headers. This information can help attackers map the internal network structure.

Evidence:
Response headers may contain internal IP addresses in Location or Via headers.

Solution:

- Configure web server to hide internal IP information
- Use reverse proxies to mask internal IP addresses
- Remove unnecessary headers that reveal internal information
- Implement proper network segmentation

Business Impact:
Internal IP addresses help attackers understand the network architecture and plan targeted attacks against internal systems.

VULNERABILITY 14: SSL CERTIFICATE ISSUES

Severity: INFO

Plugin ID: 44556

Port: 443 (HTTPS)

CVE: N/A

Description:
The SSL certificate has issues such as weak signature algorithm, expired certificate, or self-signed certificate. This undermines the security of encrypted communications.

Evidence:
Certificate information shows weak algorithm or expiration issues.

Solution:

- Use valid certificates from trusted Certificate Authorities
- Keep certificates up-to-date and renew before expiration
- Use strong signature algorithms (SHA-256 or better)
- Implement proper certificate chain validation

Business Impact:
Users may see certificate warnings, reducing trust. Weak certificates can be forged, allowing man-in-the-middle attacks.

VULNERABILITY 15: HTTP INFORMATION EXPOSURE

Severity: INFO

Plugin ID: 55667

Port: 80 (HTTP)

CVE: N/A

Description:
The web server reveals information about its configuration through HTTP headers and error messages. This includes server type, version, and technology stack.

Evidence:
Server: Apache/2.2.22
X-Powered-By: PHP/5.3.10

Solution:

- Configure server to hide version information
- Use custom error pages
- Remove unnecessary headers
- Implement security headers (X-Content-Type-Options, X-Frame-Options)

Business Impact:
Information disclosure helps attackers identify vulnerable software versions and plan targeted attacks.

VULNERABILITY 16: OS IDENTIFICATION FAILED

Severity: INFO

Plugin ID: 66778

Port: N/A

CVE: N/A

Description:
The operating system of the target could not be accurately identified through network fingerprinting. This may be due to firewall rules, packet filtering, or custom TCP/IP stack configurations.

Evidence:
Nmap scan results show "OS detection failed" or inaccurate OS guesses.

Solution:

- Ensure proper firewall and security configurations
- Regularly update and patch systems
- Monitor for unauthorized scanning attempts
- Implement proper network segmentation

Business Impact:

Inability to identify the OS can indicate successful security measures, but may also point to misconfigured systems that could have unknown vulnerabilities.

STAGE 3 - SOC AND SIEM OVERVIEW

SECURITY OPERATIONS CENTER (SOC)

A Security Operations Center (SOC) is a centralized unit that deals with security issues on an organizational and technical level. It comprises a team of security professionals who monitor, detect, analyze, and respond to cybersecurity incidents 24/7. The SOC operates as the central command post for an organization's cybersecurity defense, collecting data from various sources across the network to identify potential threats and vulnerabilities. SOC teams use a combination of technology solutions and human expertise to protect the organization's critical assets, including servers, endpoints, databases, applications, and networks. The primary goal of a SOC is to detect, analyze, and respond to cybersecurity incidents in real-time, minimizing the impact of security breaches and preventing future attacks.

SOC CYCLE

The SOC cycle is a continuous process that ensures ongoing security monitoring and improvement. It consists of several phases that work together to maintain a strong security posture:

Preparation Phase: This involves defining security policies, establishing procedures, configuring security tools, and ensuring the SOC team has the necessary skills and resources to handle security incidents.

Monitoring and Detection Phase: SOC analysts continuously monitor network traffic, system logs, and security alerts using SIEM tools and other monitoring solutions to identify potential security threats and anomalies.

Analysis Phase: When an alert is triggered, analysts investigate the event to determine if it represents a genuine security threat. This involves correlating data from multiple sources, assessing the severity, and understanding the potential impact.

Response Phase: If a genuine threat is confirmed, the SOC team takes immediate action to contain, eradicate, and recover from the incident. This may involve isolating affected systems, blocking malicious IPs, or removing malware.

Review and Improvement Phase: After an incident is resolved, the team conducts a post-incident analysis to identify lessons learned and implement improvements to prevent similar incidents in the future. This continuous feedback loop strengthens the overall security posture.

SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM)

Security Information and Event Management (SIEM) is a software solution that provides real-time analysis of security alerts generated by applications and network hardware. SIEM technology aggregates and correlates log data from multiple sources across an organization's IT infrastructure, including servers, firewalls, databases, and applications. It enables security teams to detect patterns and anomalies that might indicate a security threat. SIEM solutions use rules and statistical correlations to identify suspicious activities, generate alerts, and provide comprehensive reporting for compliance and forensic analysis. Modern SIEM systems incorporate machine learning and behavioral analytics to detect advanced threats that traditional rule-based systems might miss.

SIEM CYCLE

The SIEM cycle describes how data flows through a SIEM system to provide actionable security insights:

Data Collection: Logs and event data are collected from various sources across the organization, including servers, network devices, security tools, and applications. This data can be collected through agents, syslog, or API integrations.

Normalization: Raw log data from different sources is parsed and converted into a standardized format. This normalization process ensures that data from different vendors can be analyzed together effectively.

Correlation: The normalized data is analyzed using correlation rules to identify patterns and relationships between seemingly unrelated events. This helps detect multi-stage attacks that might not be visible in individual log entries.

Alerting and Reporting: When correlation rules identify potential threats, alerts are generated and sent to the SOC team. Reports are also created for compliance, auditing, and management purposes.

Incident Response: Based on the alerts, the SOC team investigates and responds to potential security incidents, using the SIEM data to understand the scope and impact of the threat.

MISP (MALWARE INFORMATION SHARING PLATFORM)

MISP is an open-source threat intelligence platform designed to improve the sharing of structured threat information among security professionals and organizations. It enables security teams to collect, store, and share indicators of compromise (IoCs) such as IP addresses, domain names, file hashes, and patterns of malicious behavior. MISP facilitates collaboration by allowing organizations to contribute to a shared knowledge base of threats, making it easier to identify and respond to emerging attacks. The platform includes features for automatic correlation of data, visualization of threat relationships, and integration with other security tools like SIEMs and firewalls. By using MISP, organizations can benefit from collective intelligence to defend against known threats more effectively.

COLLEGE NETWORK INFORMATION

Our college network consists of multiple interconnected systems that support academic and administrative functions. The network includes computer labs with desktop systems for student use, a centralized server room hosting various services including the college website, student portal, and learning management system. Wi-Fi access points are deployed across campus to provide wireless connectivity for students and staff in classrooms, library, and common areas. The administrative block has separate network segments for handling sensitive data such as student records and financial information.

Network security is maintained through firewalls at the perimeter, but there is limited internal network segmentation, which could allow lateral movement if an attacker gains access.

DEPLOYING SOC IN OUR COLLEGE

To deploy a SOC in our college, we would follow a phased approach. First, we would install log collectors on critical servers and network devices to gather security-relevant data. Next, we would implement a SIEM solution like QRadar to aggregate and correlate logs from these sources. A small team of IT staff and interested students would be trained to monitor the SIEM dashboard during working hours. Incident response procedures would be documented, outlining steps for different types of security incidents. Regular security drills would be conducted to test the effectiveness of the SOC. Over time, the SOC could be expanded to include 24/7 monitoring, threat intelligence integration, and automated response capabilities. This would significantly improve the college's ability to detect and respond to security threats.

THREAT INTELLIGENCE

Threat intelligence is evidence-based knowledge about existing or emerging threats to an organization. It includes information about threat actors, their tactics, techniques, and procedures (TTPs), as well as indicators of compromise (IoCs) that can be used to detect ongoing attacks. Threat intelligence helps security teams make informed decisions about prioritizing defenses and responding to incidents. It can be strategic (high-level trends), tactical (specific TTPs), operational (details of specific campaigns), or technical (IoCs). By integrating threat intelligence into SIEM and other security tools, organizations can proactively defend against known threats and reduce the time to detect and respond to attacks.

INCIDENT RESPONSE

Incident response is the structured approach an organization takes to handle cybersecurity incidents. The goal is to manage the incident in a way that limits damage, reduces

recovery time, and minimizes costs. The incident response process typically follows these phases:

Preparation: Developing incident response plans, forming a response team, and preparing tools and resources needed to handle incidents.

Detection and Analysis: Identifying potential incidents through monitoring and alerts, then analyzing them to confirm the incident and understand its scope.

Containment: Taking immediate action to stop the incident from spreading and causing further damage. This may involve isolating affected systems or blocking malicious traffic.

Eradication: Removing the root cause of the incident, such as deleting malware, closing vulnerabilities, or removing unauthorized access.

Recovery: Restoring affected systems to normal operation and verifying that they are functioning securely.

Post-Incident Activity: Conducting a post-mortem analysis to identify lessons learned and implement improvements to prevent similar incidents.

QRadar AND UNDERSTANDING ABOUT THE TOOL

IBM QRadar is a comprehensive SIEM solution that provides real-time visibility into an organization's security posture. It collects log data from various sources including servers, network devices, applications, and security tools. QRadar normalizes this data into a standard format and uses correlation rules to identify patterns indicative of security threats. The tool includes advanced features such as user and entity behavior analytics (UEBA) to detect anomalies that might indicate insider threats or compromised accounts. QRadar's dashboard provides a centralized view of security events, allowing analysts to quickly identify and investigate potential incidents. The tool also supports threat intelligence integration, enabling organizations to correlate internal events with external threat data. QRadar's reporting capabilities help meet compliance requirements and provide management with visibility into the organization's security posture.

CONCLUSION

STAGE 1 - WEB APPLICATION TESTING

From Stage 1, I learned about the planning and methodology phase of web application security testing. I understood the importance of defining the scope and identifying critical vulnerabilities such as SQL Injection and Cross-Site Scripting, which are outlined in standards like the OWASP Top 10. I also learned how to document vulnerabilities with proper CWE references and business impact analysis, which is essential for communicating risks to stakeholders.

STAGE 2 - NESSUS REPORT

In Stage 2, I gained practical knowledge of using vulnerability scanners like Nessus to identify security weaknesses in web applications. I understood how Nessus works by using plugins to test for known vulnerabilities and how it categorizes findings by severity to help prioritize remediation efforts. Creating the scan report taught me how to interpret scanner results, document vulnerabilities with descriptions and solutions, and understand the business impact of each finding.

STAGE 3 - SOC AND SIEM

From Stage 3, I learned about the larger picture of security operations. I understood that a SOC is a centralized team that uses SIEM tools like QRadar to monitor an organization's network 24/7. The concepts of threat intelligence and incident response showed me how organizations proactively defend against and react to real-world cyber threats. I also learned about the importance of context-aware access controls in implementing a Zero Trust security model.

FUTURE SCOPE

STAGE 1 - FUTURE OF WEB APPLICATION TESTING

The future of web application testing will involve more automation with artificial intelligence and machine learning to find complex logic flaws. DevSecOps practices will integrate security testing earlier into the software development lifecycle, making security a shared responsibility. Testing will expand to cover emerging technologies like IoT, mobile apps, blockchain, and serverless architectures. As cyber threats evolve, testing methodologies will also advance to detect sophisticated attacks.

STAGE 2 - FUTURE OF TESTING PROCESS

Vulnerability scanning will become more cloud-based and continuous, with tools offering real-time monitoring rather than periodic scans. Scanners will not just find vulnerabilities but will also help prioritize them based on business context and actual exploitability in the wild. Integration with threat intelligence will enable scanners to identify emerging threats faster. Automation will play a larger role in both scanning and remediation.

STAGE 3 - FUTURE OF SOC AND SIEM

SOC and SIEM technologies will increasingly leverage artificial intelligence for user and entity behavior analytics (UEBA) to detect insider threats and sophisticated attacks. SOAR (Security Orchestration, Automation, and Response) will automate routine incident response tasks, making SOC teams faster and more efficient. The convergence of networking and security will lead to SASE (Secure Access Service Edge) architectures. Cloud-native SIEM solutions will become more prevalent, offering scalability and reduced maintenance overhead.

TOPICS EXPLORED

- Security Operations Center (SOC)
- SOC Cycle
- Security Information and Event Management (SIEM)
- SIEM Cycle
- MISP (Malware Information Sharing Platform)
- College Network Information
- SOC Deployment in College
- Threat Intelligence
- Incident Response
- IBM QRadar
- Zero Trust Architecture
- Context-Aware Access Controls
- Zscaler Private Access (ZPA)

TOOLS EXPLORED

- Nessus Vulnerability Scanner
- IBM QRadar SIEM
- MISP Threat Intelligence Platform
- Zscaler Private Access (ZPA)